06
Composizione, riuso e sostituibilità: le `interface`

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Illustrare concetti generali di composizione e riuso
- Introdurre il concetto di interfaccia
- Discutere il principio di sostituibilità
- Evidenziare il polimorfismo derivante dalle interfacce

Argomenti

- Tipi di composizione e loro realizzazione
- Notazione UML
- `interface` in Java e meccanismi collegati
- Polimorfismo con le interfacce

Outline

1. Composizione e riuso
2. Interfacce
3. Tipi, sottotipi, sostituibilità, polimorfismo
4. Altri Meccanismi delle Interfacce

Intro

L'incapsulamento ci fornisce i meccanismi per ben progettare le classi, limitando il più possibile le dipendenze con chi le usa, e quindi in modo da ridurre l'impatto delle modifiche che si rendono via via necessarie.

⇒ ma le dipendenze fra classi non sono evitabili del tutto, anzi, sono un prerequisito per fare di un gruppo di classi un sistema! In più, le dipendenze sono anche manifestazione di un effettivo "riuso".

Forme di dipendenza e riuso fra classi nell'OO

- Associazione — Un oggetto ne usa un altro: "uses"
- Composizione — Un oggetto ne aggrega altri: "has-a"
- Specializzazione — Una classe ne specializza un'altra: "is-a"

Nella lezione corrente

Introdurremo la composizione (che è una versione più forte della associazione), mostrando la sua relazione con le interface di Java

Composizione – relazione "has-a"

Idea

- un oggetto della classe `A` è ottenuto componendo un insieme di altri oggetti, delle classi `B1`, `B2`,..,`Bn`
- si dice che un oggetto di `A` contiene, o si compone di, oggetti delle classi `B1`, `B2`,..,`Bn`
- ossia, lo stato dell'oggetto di `A` include le informazioni relative allo stato di un oggetto di `B1`, uno di `B2`,..,uno di `Bn`
- si noti che si parla propriamente di composizione quando `B1`, `B2`,..`Bn` non sono tipi primitivi, ma classi
- (quando gli oggetti composti hanno vita propria senza l'esistenza di `A` si parla anche di aggregazione – ma non ci occuperemo per ora in dettaglio di questa distinzione)

Qualche esempio di composizione

GUI

Un oggetto interfaccia grafica si compone di oggetti di tipo `Button`, `TextField`, `Label`, eccetera

Ateneo

Un oggetto ateneo si compone di oggetti di tipo `Facoltà`, `Studenti`, `Docenti`, eccetera

Controllore Domotica

Un oggetto controllore domotica si compone di oggetti di tipo `Lamp`, `TV`, `Radio`, eccetera

Tipiche realizzazioni

Un oggetto `A` si compone esattamente di un oggetto di `B`

- La classe `A` avrà un campo (privato) di tipo `B`
- Tale campo (impostato dal costruttore di `A`) è sempre presente

Un oggetto `A` si compone opzionalmente di un oggetto di `B`

- La classe `A` avrà un campo (privato) di tipo `B`
- Il suo contenuto potrebbe essere `null` (oggetto di `B` assente)

Un oggetto `A` si compone di un numero noto *n* di oggetti di `B`

- La classe `A` avrà *n* campi (privati) di tipo `B` – se "n" piccolo

Un oggetto `A` si compone di una moltitudine non nota di oggetti di `B`

- La classe `A` avrà un campo (privato) di tipo `B[]` (o altro container)

Ricordiamo la classe Lamp – A

```java
public class Lamp {

   /* Costanti luminosità */
   private final static int LEVELS = 10;
   private final static double DELTA = 0.1;

   /* Campi della classe */
   private int intensity;
   private boolean switchedOn;

   /* Costruttore */
   public Lamp() {
      this.switchedOn = false;
      this.intensity = 0;
   }

   /* Gestione switching */
   public void switchOn() {
      this.switchedOn = true;
   }

   public void switchOff() {
      this.switchedOn = false;
   }

   public boolean isSwitchedOn() {
      return this.switchedOn;
   }
```

Ricordiamo la classe Lamp – B

```
1     /* Gestione intensita' */
2     private void correctIntensity() { // A solo uso interno
3        if (this.intensity < 0) {
4           this.intensity = 0;
5        } else if (this.intensity > LEVELS) {
6           this.intensity = LEVELS;
7        }
8     }
9
10    public void setIntensity(final double value) {
11       this.intensity = Math.round((float) (value / DELTA));
12       this.correctIntensity();
13    }
14
15    public void dim() {
16       this.intensity--;
17       this.correctIntensity();
18    }
19
20    public void brighten() {
21       this.intensity++;
22       this.correctIntensity();
23    }
24
25    public double getIntensity() {
26       return this.intensity * DELTA;
27    }
28
```

Un esercizio: dispositivo `TwoLampsDevice`

Caratteristiche del sistema da modellare

- una base su cui poggiano due lampadine
- possibilità di accendere/spegnere entrambe
- possibilità di modalità "eco"

Idea realizzativa 1 :(

- una classe con 4 campi, ossia le due intensità e i due flag
- sarebbe un buon design?
- riuserei codice? starei aderendo al principio DRY? (Don't Repeat Yourself)

Idea realizzativa 2 :)

- riuso `Lamp` e sfrutto la composizione

Esempio: TwoLampsDevice pt 1

```java
public class TwoLampsDevice {

  /* Composizione immutabile di due Lamp! */
  private final Lamp l1;
  private final Lamp l2;

  /* Composizione inizializzata al momento della costruzione */
  public TwoLampsDevice() {
    this.l1 = new Lamp();
    this.l2 = new Lamp();
  }

  /* Metodi getter */
  public Lamp getFirst() {
    return this.l1;
  }

  public Lamp getSecond() {
    return this.l2;
  }
```

Esempio: TwoLampsDevice pt 2

```
  /* Altri metodi */
  public void switchOnBoth() {
    this.l1.switchOn();
    this.l2.switchOn();
  }

  public void switchOffBoth() {
    this.l1.switchOff();
    this.l2.switchOff();
  }

  public void ecoMode() {
    this.l1.switchOff();
    this.l2.switchOn();
    this.l2.setIntensity(0.5);
  }


}
```

La necessità di una notazione grafica – UML

Here!

UML – Unified Modelling Language

- È un linguaggio grafico e OO-based per modellare software
- Facilita lo scambio di documentazione, e il ragionamento su sistemi articolati e complessi
- È uno standard dell'OMG dal 1996
- È molto utile anche a fini didattici
- Noi ne useremo solo la parte chiamata Class Diagram
- Nel corso di Ingegneria del Software lo approfondirete

Class Diagram

..diagramma delle classi, una prima descrizione

- Un box rettangolare per classe, diviso in tre aree:
  - 1. nome della classe, 2. campi, 3. metodi (e costruttori)
- Su campi e metodi
  - si antepone il simbolo `-` se privati, `+` se pubblici
  - si sottolineano se `static`
  - dei metodi si riporta solo la signature, con sintassi:
    `nome(arg1: tipo1, arg2: tipo2, ..): tipo_ritorno`
- archi fra classi indicano relazioni speciali:
  - con rombo (composizione), con freccia (semplice associazione)
  - con triangolo (generalizzazione)
  - l'arco può essere etichettato con la molteplicità (1, 2, 0..1, 0..n, 1..n)

A seconda dello scopo per cui si usa il diagramma, non è necessario riportare tutte le informazioni, ad esempio spesso si omettono le proprietà, le signature complete, ed alcune relazioni

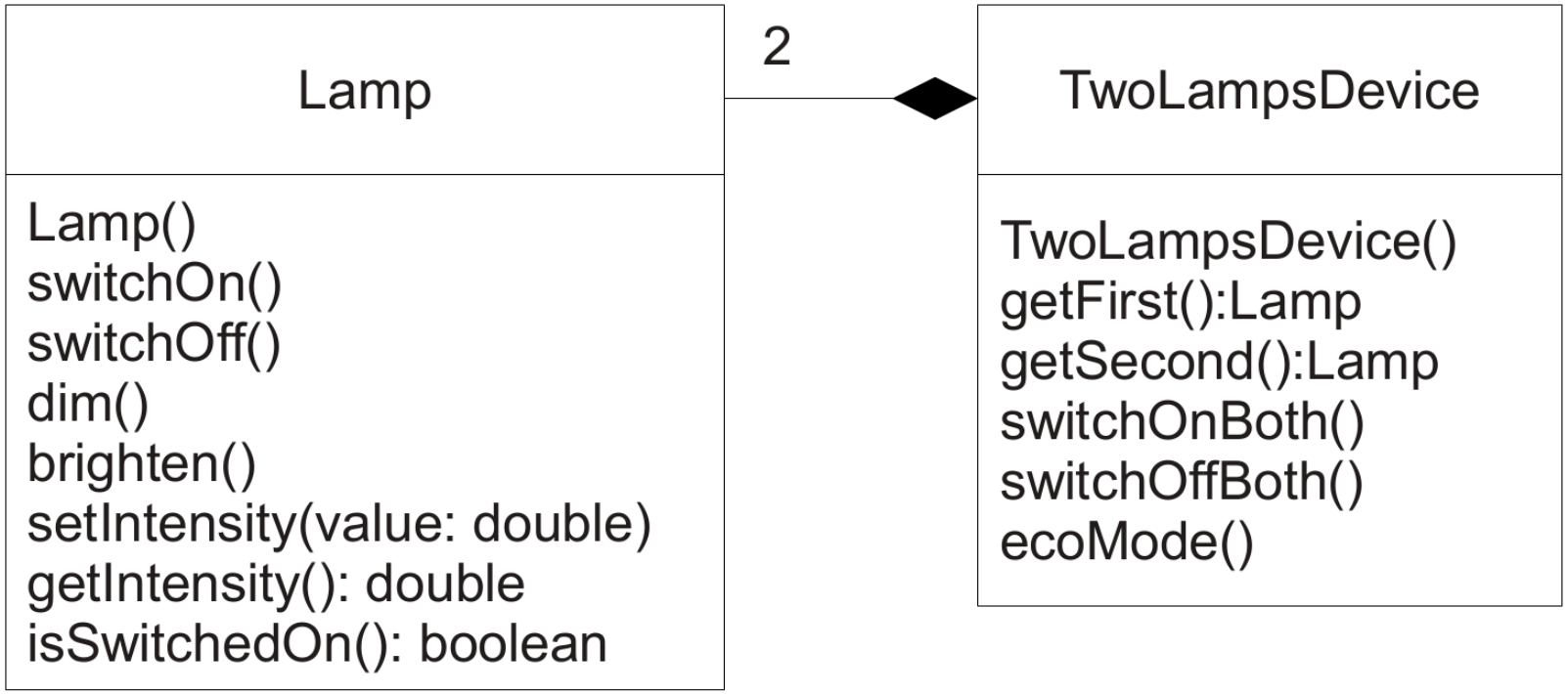
Notazione UML completa per la classe `Lamp`: tipicamente usata in fase di implementazione

Lamp
- LEVELS: int - DELTA: double - intensity: int - switchedOn: boolean
+ Lamp() + switchOn() + switchOff() + isSwitchedOn(): boolean - correctIntensity() + setIntensity(value: double) + dim() + brighten() + getIntensity(): double

Notazione parziale: solo parte pubblica: tipicamente usata in fase di design

Lamp
Lamp() switchOn() switchOff() dim() brighten() setIntensity(value: double) getIntensity(): double isSwitchedOn(): boolean

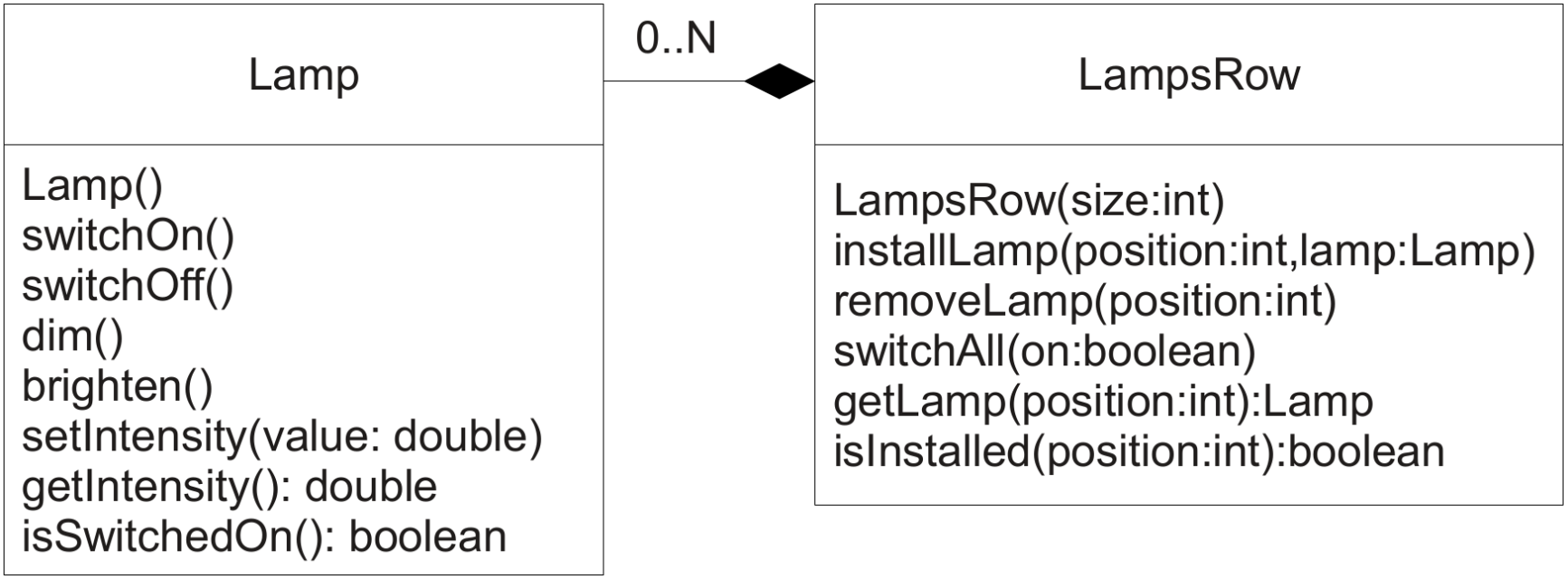
UML: Lamp e TwoLampsDevice



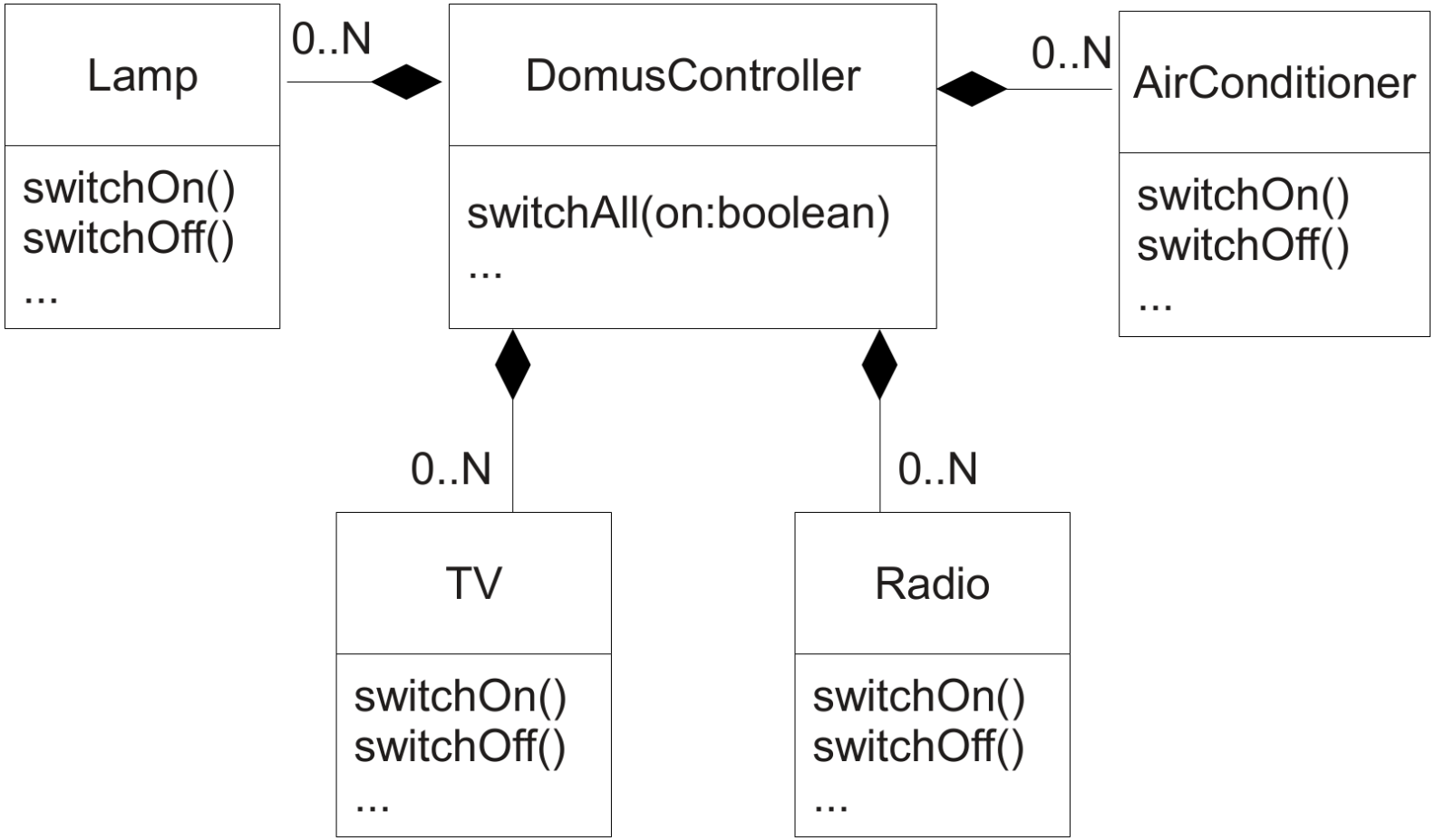
Altro caso di composizione: LampsRow

```java
public class LampsRow{

    private final Lamp[] row; // Campo

    public LampsRow(final int size){
      this.row = new Lamp[size]; // Tutti i valori a null
    }
    public void installLamp(final int position, final Lamp lamp){
      this.row[position] = lamp;
    }
    public void removeLamp(final int position){
      this.row[position] = null;
    }
    public void switchAll(final boolean on){
      for (Lamp lamp: this.row){
          if (lamp != null){ // Previene il NullPointException
            if (on){
                lamp.switchOn();
            } else {
                lamp.switchOff();
            }
          }
      }
    }
    public Lamp getLamp(final int position){ // Selettore
      return this.row[position];
    }
    public boolean isInstalled(final int position){ // Selettore
      return this.row[position] != null;
   }
}
```

UML: Lamp e LampsRow



Scenario DomusController



Come scrivereste il metodo `switchAll` in modo riusabile, e possibilmente rimandendo aperti all'introduzione di nuovi tipi di dispositivi?

Realizzazione senza riuso: schema

```
public class DomusController{

    private Lamp[] lamps;
    private TV[] tvs;
    private AirConditioner[] airs;
    private Radio[] radios;

    ...
    public void switchAll(boolean on){
      for (Lamp lamp: this.lamps){
          if (lamp != null){
            if (on){
                lamp.switchOn();
            } else {
                lamp.switchOff();
            }
          }
      }
      for (TV tv: this.tvs){
          if (tv != null){
            if (on){
                tv.switchOn();
            } else {
                tv.switchOff();
            }
          }
      }
      ... // e così via per tutti i dispositivi
    }

}
```

Outline

1. Composizione e riuso
2. Interfacce
3. Tipi, sottotipi, sostituibilità, polimorfismo
4. Altri Meccanismi delle Interfacce

Motivazioni

Specifica

- Serve un meccanismo per separare esplicitamente, ossia in dichiarazioni diverse, l'interfaccia della classe e la sua realizzazione
- Questo consente di tenere separate fisicamente la parte di "contratto" (tipicamente fissa) con quella di "implementazione" (modificabile frequentemente)
- Consente di separare bene il progetto dall'implementazione

Polimorfismo

- Serve un meccanismo per poter fornire diverse possibili realizzazioni di un contratto
- Tutte devono poter essere utilizzabili in modo omogeneo
- Nel caso di `DomusController`:
  - Avere un unico contratto per i "dispositivi", e..
  - ..diverse classi che lo rispettano
  - `DomusController` gestirà un unico array di "dispositivi"

Java interfaces

Cos'è una interface

- È un nuovo tipo di dato dichiarabile (quindi come le classi)
- Ha un nome, e include "solo" un insieme di intestazioni di metodi
- Viene compilato da `javac` come una classe, e produce un `.class`

Una interface I può essere "implementata" da una classe

- Attraverso una classe `C` che lo dichiara esplicitamente (`class C implements I{..}`)
- `C` dovrà definire (il corpo di) tutti i metodi dichiarati in `I`
- Un oggetto istanza di `C`, avrà come tipo `C` al solito, ma anche `I`!!

Esempio: dispositivi DomusController

`Lamp`, `TV`, `Radio`, `AirConditioner` hanno una caratteristica comune, sono dispositivi e come tali possono come minimo essere accesi o spenti. È possibile definire una interfaccia `Device` che tutti e 4 implementano.

Interface Device

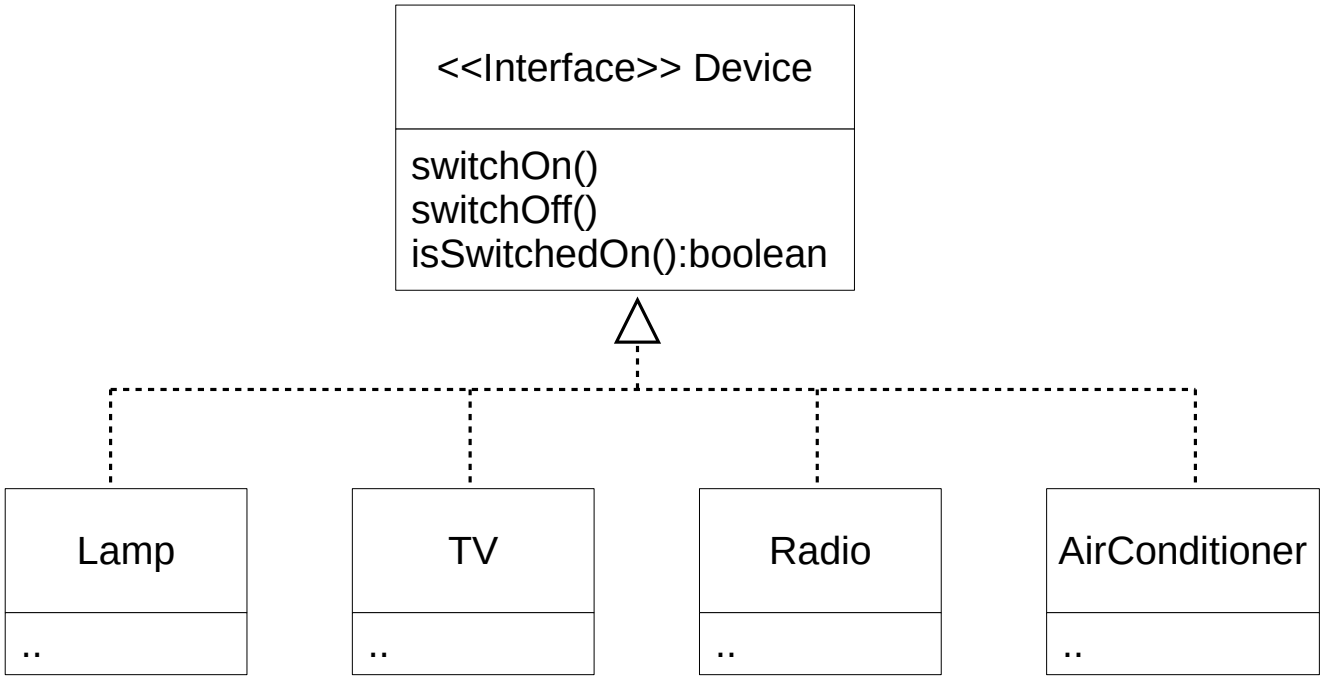
```
/* Interfaccia per dispositivi
Definisce un contratto:
- si può accendere
- si può spegnere
- si può verificare se acceso/spento

Nota: nessuna indicazione
      public/private nei metodi!
*/

public interface Device {

  void switchOn();

  void switchOff();

  boolean isSwitchedOn();
}
```

Implementazioni di Device

```
public class Lamp implements Device{
    ...  // Nessun cambiamento necessario rispetto a prima!
    private boolean switchedOn;

    public void switchOn(){
      this.switchedOn = true;
    }
    public void switchOff(){
      this.switchedOn = false;
    }
    public boolean isSwitchedOn(){
      return this.switchedOn;
    }
    ...
}

public class TV implements Device{
    ... // Nessun cambiamento necessario rispetto a prima!
}

public class Radio implements Device{
    ... // Nessun cambiamento necessario rispetto a prima!
}
```

Notazione UML per le interfacce

- interfaccia come box con titolo "`<< interface >> Nome`"
- arco tratteggiato (punta a triangolo) per la relaz. "`implements`"
- archi raggruppati per migliorare la resa grafica



Interfacce come tipi di dato

Data l'interfaccia `I`, in che senso `I` è un tipo?

- `I` è un tipo come gli altri (`int`, `float`, `String`, `Lamp`, `Lamp[]`)
- è usabile per dichiarare variabili, come tipo di input/output di una funzione, come tipo di un campo

Quali operazioni sono consentite?

esattamente (e solo) le chiamate dei metodi definiti dall'interfaccia

Quali sono i valori (/oggetti) di quel tipo?

gli oggetti delle classi che dichiarano implementare quell'interfaccia

Interfacce e assegnamenti

```
/* Su Lamp effettuo le usuali operazioni */
Lamp lamp = new Lamp();
lamp.switchOn();
boolean b = lamp.isSwitchedOn();

/* Una variabile di tipo Device può contenere
   un Lamp, e su essa posso eseguire le
   operazioni definite da Device */
Device dev;    // creo la variabile
dev = new Lamp(); // assegnamento ok
dev.switchOn();    // operazione di Device
boolean b2 = dev.isSwitchedOn();  // operazioni di Device

Device dev2 = new Lamp();  // Altro assegnamento

/* Attenzione, le interfacce non sono istanziabili */
// Device dev3 = new Device(); NO!!!!
```

Ridurre la molteplicità / aumentare riuso – prima

```
class DeviceUtilities{

    /* Senza interfacce, non resta altro che... */

    public static void switchOnIfCurrentlyOff(Lamp lamp){
      if (!lamp.isSwitchedOn()){
          lamp.switchOn();
      }
    }

    public static void switchOnIfCurrentlyOff(TV tv){
      if (!tv.isSwitchedOn()){
          tv.switchOn();
      }
    }

    public static void switchOnIfCurrentlyOff(Radio radio){
      if (!radio.isSwitchedOn()){
          radio.switchOn();
      }
    }
    ..
}
```

Ridurre la molteplicità / aumentare riuso – dopo

```
class DeviceUtilities{

    /* Grazie alle interfacce, fattorizzo in un solo metodo */

    public static void switchOnIfCurrentlyOff(Device device){
      if (!device.isSwitchedOn()){
          device.switchOn();
      }
    }
}

/* Codice cliente */
Lamp lamp = new Lamp();
TV tv = new TV();
Radio radio = new Radio();

switchOnIfCurrentlyOff(lamp); // OK, un Lamp è un Device
switchOnIfCurrentlyOff(tv); // OK, una TV è un Device
switchOnIfCurrentlyOff(radio); // OK, una Radio è un Device
```

Razionale delle interfacce

Quando costruire una interfaccia?

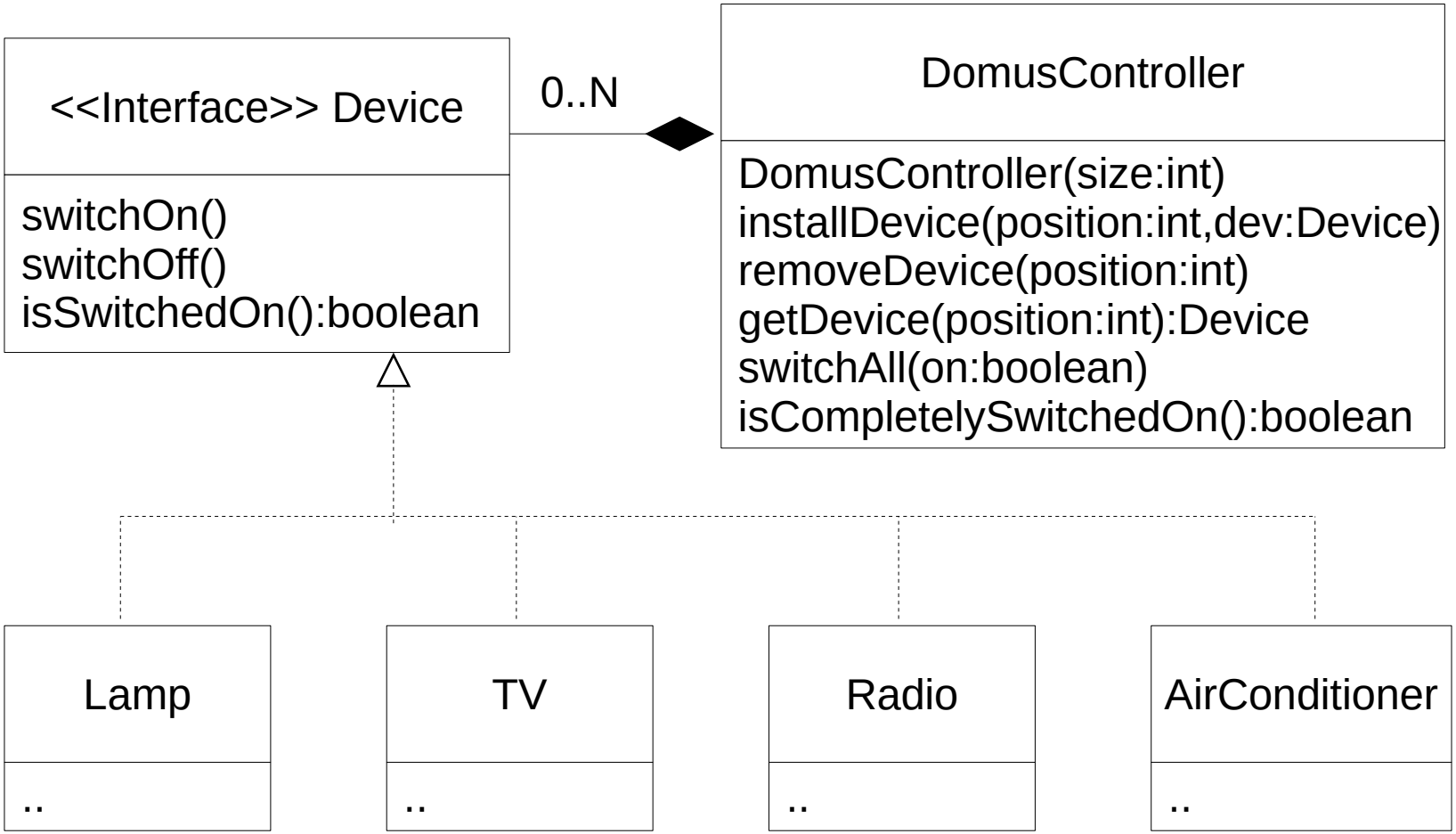
- quando si ritiene utile separare contratto da implementazione (sempre vero per i concetti cardine in applicazioni complesse)
- quando si prevede la possibilità che varie classi possano voler implementare un medesimo contratto
- quando si vogliono costruire funzionalità che possano lavorare con qualunque oggetto che implementi il contratto
  - caso particolare: Quando si vuole comporre ("has-a") un qualunque oggetto che implementi il contratto

⇒ l'esperienza mostra che classi riusabili di norma hanno sempre una loro `interface`

Quindi:

- laddove ci si aspetta un oggetto che implementi il contratto si usa il tipo dell'interfaccia
- questo consente il riuso della funzionalità a tutte le classi che implementano il contratto

Scenario DomusController rivisitato



Codice DomusController pt 1

```java
public class DomusController {

  /* Compongo n oggetti che implementano Device */
  private final Device[] devices;

  public DomusController(final int size) {
    this.devices = new Device[size];
  }

  /* Aggiungo un device */
  public void installDevice(final int position, final Device dev) {
    this.devices[position] = dev;
  }

  /* Rimuovo un device */
  public void removeDevice(final int position) {
    this.devices[position] = null;
  }

  public Device getDevice(final int position) {
    return this.devices[position];
  }
```

Codice DomusController pt 2

```java
  /* Spengo/accendo tutti i device */
  public void switchAll(final boolean on) {
    for (Device dev : this.devices) {
      if (dev != null) { // Prevengo il NullPointerException
        if (on) {
          dev.switchOn();
        } else {
          dev.switchOff();
        }
      }
    }
  }

  /* Verifico se sono tutti accesi */
  public boolean isCompletelySwitchedOn() {
    for (Device dev : this.devices) {
      if (dev != null && !dev.isSwitchedOn()) {
        return false;
      }
    }
    return true;
  }
}
```

Codice TV

```java
public class TV implements Device {

  /* Campi della classe */
  private boolean switchedOn;

  /* Costruttore */
  public TV() {
    this.switchedOn = false;
  }

  /* Metodi per accendere e spegnere */
  public void switchOn() {
    this.switchedOn = true;
  }

  public void switchOff() {
    this.switchedOn = false;
  }

  public boolean isSwitchedOn() {
    return this.switchedOn;
  }

  public String toString() {
    return "I'm a TV..";
  }
}
```

Uso di DomusController

```
public class UseDomusController{

    public static void main(String[] s){
      // Creo un DomusController
      final DomusController dc = new DomusController(10);

      // Installo dei dispositivi
      dc.installDevice(0,new Lamp());
      dc.installDevice(1,new Lamp());
      dc.installDevice(2,new Lamp());
      dc.installDevice(3,new TV());
      dc.installDevice(4,new TV());
      dc.installDevice(5,new Radio());

      // Li accendo tutti
      dc.switchAll(true);

      // Verifico l'accensione
      final boolean b = dc.isCompletelySwitchedOn(); // true
    }
}
```

Outline

1. Composizione e riuso
2. Interfacce
3. Tipi, sottotipi, sostituibilità, polimorfismo
4. Altri Meccanismi delle Interfacce

`implements` come relazione di "sottotipo"

Un tipo è considerabile come un set di valori/oggetti

- $T_{\texttt{boolean}} = \{\texttt{true}, \texttt{false}\}$
- $T_{\texttt{int}} = \{-2147483648, \ldots, -1, 0, 1, 2, \ldots, 2147483647\}$
- $T_{\texttt{Lamp}} = \{X | X \text{ is an object of class } \texttt{Lamp}\}$
- $T_{\texttt{Device}} = \{X | X \text{ is an object of a class implementing } \texttt{Device}\}$
- $T_{\texttt{String}} = \{X | X \text{ is an object of class } \texttt{String}\}$

`Lamp` è un sottotipo di `Device`!

- Un oggetto della classe `Lamp` è anche del tipo `Device`
- Quindi, da $X \in T_{\texttt{Lamp}}$ segue $X \in T_{\texttt{Device}}$
- Ossia, $T_{\texttt{Lamp}} \subseteq T_{\texttt{Device}}$, scritto anche: `Lamp <: Device`

Ogni classe è sottotipo delle interfacce che implementa!

Sottotipi e principio di sostituibilità

Principio di sostituibilità di Liskov (1993)

Se `A` è un sottotipo di `B` allora ogni oggetto (o valore) di `A` deve essere utilizzabile dove un programma si attende un oggetto (o valore) di `B`

Nel caso delle interfacce

Se la classe `C` implementa l'interfaccia `I`, allora ogni istanza di `C` può essere passata dove il programma si attende un elemento del tipo `I`.

Si rischiano errori?

No. Il programma può manipolare gli elementi del tipo `I` solo mandando loro i messaggi dichiarati in `I`, che sono sicuramente "accettati" dagli oggetti di `C`. Il viceversa non è vero.

Nota: `I` è più generale di `C`, ma fornisce meno funzionalità!

Polimorfismo

Polimorfismo = molte forme (molti tipi)

Ve ne sono di diversi tipi nei linguaggi OO

- Polimorfismo inclusivo: subtyping
- Polimorfismo parametrico: genericità

Polimorfismo inclusivo

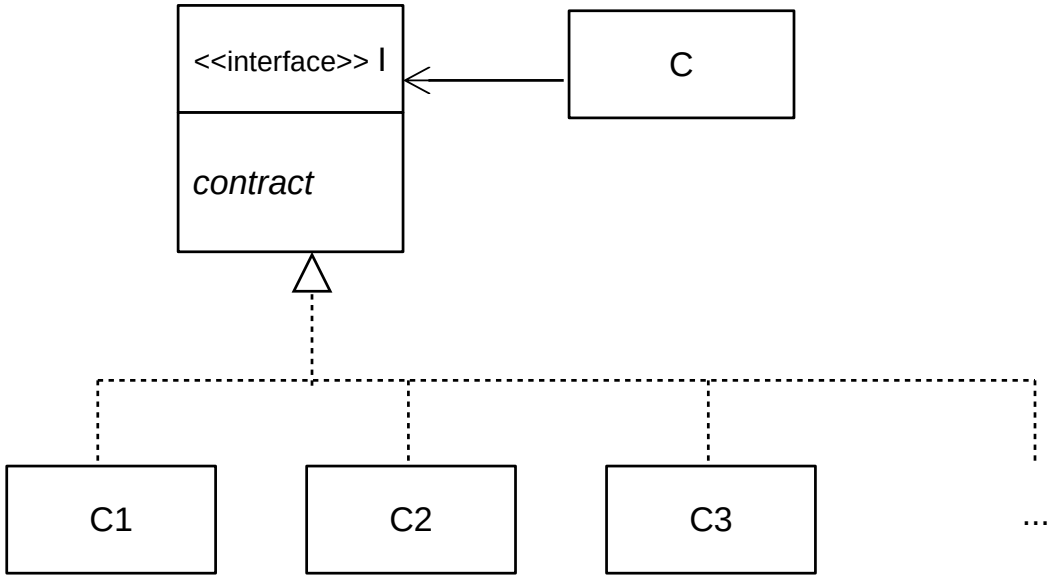
È precisamente l'applicazione del principio di sostituibilità

- Se il tipo `A` è una specializzazione di `B` (lo implementa)..
- ..si può usare un oggetto `A` dove se ne attende uno di `B`

Polimorfismo e interfacce

Una delle pietre miliari dell'OO

- `C` deve usare uno o più oggetti di un tipo non predeterminato
- l'interfaccia `I` cattura il contratto comune di tali oggetti
- varie classi `C1`,`C2`,`C3` (e altre in futuro) implementano `I`
- `C` non ha dipendenze rispetto `C1`,`C2`,`C3`
- (l'uso potrebbe essere una composizione, come nel caso precedente)



Late binding (o dynamic binding)

```
1 public static void switchOnIfCurrentlyOff(Device device){
2     // Collegamento dinamico con i metodi da invocare..
3     if (!device.isSwitchedOn()){
4         device.switchOn();
5     }
6 }
7 /* Codice cliente */
8 Lamp lamp = new Lamp();
9 switchOnIfCurrentlyOff(lamp); // OK, un Lamp è un Device
```

Collegamento ritardato

Accade con le chiamate a metodi non-statici

- Dentro a `switchOnIfCurrentlyOff()` mandiamo a `device` due messaggi (`isSwitchedOn` e `switchOn`), ma il codice da eseguire viene scelto dinamicamente (ossia "late"), dipende dalla classe dell'oggetto `device` (`Lamp`,`TV`,..)
- Terminologia: `device` ha tipo `Device` (tipo statico), ma a tempo di esecuzione è un `Lamp` (tipo run-time)

Early (static) vs late (dynamic) binding

```
interface I {
    void m();
}

class C1 implememts I {
    void m(){ System.out.println("I'm istance of C1");}
}

class C2 implememts I {
    void m(){ System.out.println("I'm istance of C2");}
    static void m2(){ System.out.println("I'm a static method of C2");}
}

// Codice cliente

I i = Math.random() > 0.5 ? new C1() : new C2();
i.m(); // collegamento al body da eseguire è late, ossia dinamico

C2.m2(); // collegamento al body da eseguire è early, ossia statico
```

Differenze

- Early: con metodi statici o finali
- Late: negli altri casi

Outline

1. Composizione e riuso
2. Interfacce
3. Tipi, sottotipi, sostituibilità, polimorfismo
4. Altri Meccanismi delle Interfacce

Altri aspetti sulle interfacce

Cosa non può contenere una `interface`?

- Non può contenere campi istanza
- Non può contenere il corpo di un metodo istanza
- Non può contenere un costruttore

Cosa potrebbe contenere una `interface`?

- Java consentirebbe anche di includere dei campi statici e metodi statici (con tanto di corpo), ma è sconsigliato utilizzare questa funzionalità per il momento
- In Java 8 i metodi possono avere una implementazione di default, che vedremo

Le `interface` includano solo intestazioni di metodi!

Implementazione multipla

Implementazione multipla

Dichiarazione possibile: `class C implements I1,I2,I3{..}`

- Una classe `C` implementa sia `I1` che `I2` che `I3`
- La classe `C` deve fornire un corpo per tutti i metodi di `I1`, tutti quelli di `I2`, tutti quelli di `I3`
  - se `I1`,`I2`,`I3` avessero metodi in comune non ci sarebbe problema, ognuno andrebbe implementato una volta sola
- Le istanze di `C` hanno tipo `C`, ma anche i tipi `I1`, `I2` e `I3`

Esempio Device e Luminous

```java
/* Interfaccia per dispositivi */
public interface Device{  // Contratto:
    void switchOn();     // - si può accendere
    void switchOff();    // - si può spegnere
    boolean isSwitchedOn(); // - si può verificare se acceso/spento
}

/* Interfaccia per entità luminose */
public interface Luminous{       // Contratto:
    void dim();      // - si può ridurre la luminosità
    void bright();     // - si può aumentare la luminosità
}

public class Lamp implements Device, Luminous{
    ...
    public void switchOn(){ ... }
    public void switchOff(){ ... }
    public boolean isSwitchedOn(){ ... }
    public void dim(){ ... }
    public void bright(){ ... }
}
```

Estensione interfacce

Estensione

Dichiarazione possibile: `interface I extends I1,I2,I3{..}`

- Una interfaccia `I` definisce certi metodi, oltre a quelli di `I1`, `I2`, `I3`
- Una classe `C` che deve implementare `I` deve fornire un corpo per tutti i metodi indicati in `I`, più tutti quelli di `I1`, tutti quelli di `I2`, e tutti quelli di `I3`
- Le istanze di `C` hanno tipo `C`, ma anche i tipi `I`, `I1`, `I2` e `I3`

Esempio LuminousDevice

```
public interface Device{
    void switchOn();
    void switchOff();
    boolean isSwitchedOn();
}
public interface Luminous{
    void dim();
    void bright();
}

/* Interfaccia per dispositivi luminosi */
public interface LuminousDevice extends Device, Luminous{
    // non si aggiungono ulteriori metodi
}

public class Lamp implements LuminousDevice{
    ...
    public void switchOn(){ ... }
    public void switchOff(){ ... }
    public boolean isSwitchedOn(){ ... }
    public void dim(){ ... }
    public void bright(){ ... }
}
```

Qualche esempio dalle librerie Java

Interfacce base

- `java.lang.Appendable`
- `java.io.DataInput`
- `java.io.Serializable`: interfaccia "tag"
- `javax.swing.Icon`

Implementazioni multiple

- `class ImageIcon implements Icon, Serializable, Accessible{...}`

Estensioni di interfacce

- `interface ObjectInput extends DataInput{...}`

Preview del prossimo laboratorio

Obbiettivi

Familiarizzare con:

- Costruzione di semplici classi con incapsulamento
- Relativa costruzione di test
- Costruzione di classi con relazione d'uso verso una interfaccia